

Redspher

Full Stack Developer

Technical interview prep - mobile PDF

React + TypeScript | Node.js + REST

AWS + CI/CD | AI / MCP / Agents

Goal

This version is focused on the technical interview: answer outlines, trade-offs, mini system design scenarios, coding discussion points, and architecture questions.

How to use it

- Read the Priority page first.
- Practise the Q&A cards out loud.
- For system design pages, explain the architecture in 2-3 minutes.
- Always connect answers to one concrete example from your own experience.

Source: the uploaded Redspher job description. This is not an official question list.

Contents

- 1. What they will likely measure
- 2. 60-second technical introduction
- 3. TypeScript and React questions
- 4. Node.js, REST API, error handling
- 5. AWS, CI/CD, observability
- 6. AI / MCP / agent-based solutions
- 7. Mini system design scenarios
- 8. Legacy PHP integration
- 9. Smart questions to ask them
- 10. Last-day review

Technical focus inferred from the JD

- Business-oriented full-stack development.
- React + TypeScript frontend and backend services.
- AWS cloud environment, REST APIs, Git, CI/CD.
- AI-driven solutions: MCP, AI agents, orchestration, automation.
- PHP is a plus for legacy maintenance or integration.

1. What they will likely measure

Not just textbook knowledge

Based on the JD, the important question is whether you can turn a business need into a working, maintainable full-stack solution, while thinking clearly about new AI and automation directions.

React/TS	Clean components, state choices, effects, performance, type safety.
Backend	REST, validation, auth, error handling, async workflows.
AWS	Pragmatic deployment, IAM, monitoring, managed services.
Architecture	Trade-offs: delivery speed vs scale, simplicity vs flexibility.
AI/MCP	Not hype: security, tool calling, audit, human approval, evals.
Business	Logistics/pricing domain: accuracy, latency, data quality, audit.

Best positioning

I am a full-stack developer with a strong React/TypeScript background, but I am also comfortable with backend integrations, observability and automation. I choose technology to solve business problems, not for its own sake.

2. 60-second technical intro

Do not read this word for word. Learn the structure and say it naturally.

Answer outline

I am a full-stack developer with strong React and TypeScript experience, and I have also worked on Node.js based backend and real-time systems. In previous roles I built business-facing applications, improved frontend architecture, integrated APIs, worked with monitoring and release processes, and created automation around internal workflows. What I like about this role is that it combines practical full-stack product work with AWS and new AI/automation topics. I am especially interested in building useful AI-assisted workflows, but with proper security, observability and human control.

If they ask about AWS

I would not present myself as a senior AWS architect, but I have hands-on AWS learning and project experience. I understand the main building blocks: Lambda/API Gateway, IAM, S3, DynamoDB/RDS trade-offs, queues, CloudWatch, and deployment basics. I am comfortable learning the exact company setup quickly.

If they ask about AI

I would keep it pragmatic: identify a narrow business workflow, connect the model to safe tools and relevant context, add guardrails, logging, human approval for risky actions, and measure quality before scaling it.

3. TypeScript - what to know

Main message

TypeScript is not just JavaScript with types. In an interview, show that you use type safety to reduce runtime bugs, clarify API contracts, and make refactoring safer.

TS

What is the difference between any and unknown?

any disables type checking. unknown is safer: you must narrow it before use. For external APIs, JSON parsing or user input, unknown + runtime validation is better.

TS

When would you use a union type?

When a value can be in a few well-defined states. Example: loading | success | error. With a discriminated union, UI states become safer and easier to reason about.

TS

What is the purpose of generics?

Reusable but type-safe code. Example: an API client can receive the expected response type as a type parameter, so the caller keeps precise typing.

TS

How would you handle backend DTOs in TypeScript?

Separate request and response DTOs, runtime schema validation, and where possible shared contracts or OpenAPI-generated types. TypeScript is compile-time only; it does not validate runtime input.

4. React - likely questions

Main message

Do not just list hook names. Show that you know what should be local state, server state, derived state, and when performance optimization is actually needed.

REACT

When would you use useEffect?

For side effects: data loading triggers, subscriptions, browser APIs, or integration outside React. I would not use it for simple derived state; that belongs in render logic or useMemo if needed.

REACT

What is a controlled component?

The input value is controlled by React state. Benefits: validation, conditional UI, clearer submit logic. Downsides: boilerplate and possible performance issues in very large forms.

REACT

When do you use memo/useMemo/useCallback?

When there is a measured re-rendering problem or a real need for stable references. Not by default everywhere, because it increases complexity.

REACT

How would you handle server state?

Cache, loading/error state, refetching and invalidation. In a large app I prefer a dedicated pattern or library, not random useEffect calls everywhere.

React mini practical tasks

1. API table with filters

- State: query, filters, pagination, selected row.
- Server state: loading/error/data, cache, refetch.
- UX: debounced search, empty state, retry, skeleton.
- Types: map DTOs to UI models.

2. Pricing form

- Controlled form or form library with schema validation.
- Backend-aligned error mapping.
- Optimistic UI only if rollback is simple.
- Money, currency and rounding must be handled carefully.

3. Realtime status dashboard

- Polling, SSE or WebSocket trade-offs.
- Reconnect strategy and stale-data indicator.
- Backpressure: do not render every event immediately.
- Observability: client errors and latency metrics.

Your own example

Bring in your real-time/WebSocket and React/TypeScript experience: live data, API integration, release work, monitoring, or production support.

5. Node.js backend

Main message

Node.js is strong for I/O-bound backends, APIs and integrations. In the interview, focus on the event loop, async error handling, structure, validation and observability.

NODE

How does the Node event loop work, briefly?

JavaScript execution is single-threaded, while I/O is handled asynchronously through libuv/OS mechanisms. CPU-heavy work can block the event loop; use worker threads, queues or a separate service.

NODE

How do you structure a backend service?

Controller for HTTP input/output, service for business logic, repository/adapters for external systems, validation layer, central error handling, separate config and logging.

NODE

How do you handle async errors?

try/catch in async functions, central error middleware, typed/domain errors, structured logs, correlation id, and timeout/retry/circuit breaker for external APIs when justified.

NODE

When would you use a queue?

When the process is slow, waits for an unreliable external system, is retryable, or does not need a synchronous response. Examples: pricing recalculation, email, import/export, AI batch analysis.

6. REST API design

Main message

REST interview questions are not just definition hunting. The key is resource modelling, HTTP semantics, errors, auth, idempotency, pagination and evolvability.

Status codes	200/201/204, 400 validation, 401 auth, 403 permission, 404, 409 conflict, 422 domain validation, 429 rate limit, 5xx server errors.
Idempotency	GET/PUT/DELETE are normally idempotent. For risky POST operations, use an idempotency key.
Pagination	Offset is simple. Cursor is more stable for large or frequently changing data.
Versioning	URI or header can work. More important: backwards compatibility, deprecation, contract tests.
Errors	Stable error code + message + details + request id. Never leak stack traces.

REST

How would you design a shipment tracking API?

Resources: /shipments, /shipments/{id}, /shipments/{id}/events. Lists need filters and cursor pagination. Events can be immutable; current status can be derived or denormalized.

REST

Is adding an enum value a breaking change?

It can be breaking for consumers using exhaustive switches and no unknown fallback. Document the contract and make clients tolerant of unknown values.

7. AWS at full-stack level

Do not overplay it

Say that you bring pragmatic fundamentals and hands-on learning. In a full-stack interview, good service choice, IAM, observability and release thinking matter more than memorising every AWS service.

Static frontend	S3 + CloudFront, CI/CD deploy, cache invalidation.
REST backend	Lambda + API Gateway for smaller services, or ECS/Fargate for long-running backends.
Async work	SQS/EventBridge, DLQ, retry, idempotent worker.
Relational data	RDS/Aurora, migrations, backups, indexes.
Key-value scale	DynamoDB, partition key design, access patterns first.
Monitoring	CloudWatch logs/metrics/alarms, tracing, dashboards, correlation id.
Secrets	Secrets Manager or SSM Parameter Store; no hardcoded secrets.

Reliability talking points

- Timeout on every external call.
- Retry with exponential backoff, only when the operation is safe/idempotent.
- DLQ and alerts so failed messages do not disappear.
- Health checks, alarms, rollback and runbooks.
- Business metrics: quote success rate, shipment event freshness, API latency.

AWS Q&A

AWS

Lambda or container?

Lambda is good for event-driven, short, bursty workloads. A container is better for long-running processes, custom dependencies, stable connections or more runtime control. The workload should decide.

AWS

RDS or DynamoDB?

RDS is good for complex relationships, SQL queries and transactions. DynamoDB is good at high scale with known access patterns. In logistics/pricing, you may use both: transactional core plus denormalized read model.

AWS

What do you watch with IAM?

Least privilege, role-based access, separate environments/accounts, secrets outside code, audit logs. All tools need especially narrow scope.

AWS

How would you deploy safely?

Build-test-scan, staging, migration plan, canary/blue-green where justified, feature flag, health checks, monitored rollout, fast rollback.

AWS

What does observability mean?

Logs, metrics and traces together. Add request ids, domain events, error rate, latency percentiles and business success metrics.

8. Git and CI/CD

Interview answer

CI/CD is not just having a pipeline. It means small reviewed changes can reach production reliably, with tests and a realistic rollback path.

Lint/typecheck	Catch cheap errors early.
Unit tests	Fast checks for business logic and components.
Integration tests	API, DB and external adapter contracts.
Build artifact	Same artifact goes to staging and production.
Security scan	Dependency and secret issues.
Deploy	Staging -> approval -> production.
Rollback	Versioned artifact, feature flag, migration rollback plan.

What I check in code review

- Does it solve the business problem?
- Is it readable and testable?
- Is there input validation and error handling?
- Could it cause a security or performance regression?
- Is it easy to monitor or roll back?

9. AI / MCP / agents - technical view

Main message

The best answer is not hype. The value is connecting AI safely to a real business workflow, with tools, permissions, audit, evaluation and human control.

LLM	Text and reasoning assistant. Not a deterministic business rule engine.
Agent	LLM + goal + tool use + multi-step reasoning/execution.
Tool calling	The model calls an external function/API with structured parameters.
MCP	A standard protocol between LLM apps and external context/tool sources.
RAG	Retrieve relevant documents/data before answering. Not the same as fine-tuning.
Orchestration	Steps, tools, state, retries, approvals and audit tied together.

MCP essentials

- Server features: resources, prompts and tools.
- Tools: model-invokable functions for external systems.
- Resources: context/data such as files, schemas or domain data.
- Security: input validation, access control, rate limit, output sanitization, user confirmation for sensitive actions.

AI agent Q&A

AI

How would you build an AI assistant for a logistics team?

Start with a narrow scope: search and summarise shipment/pricing data. Use RAG/context in read-only mode first, then safe tool actions later. Add auth, audit, PII guardrails and human approval for writes.

AI

What are the risks of agents?

Non-deterministic decisions, hallucination, prompt injection, unauthorized data access, tool misuse and data exfiltration. Use narrow permissions, allowlisted tools, approval, logging and evaluation.

AI

How would you measure quality?

Golden test set, human review, task-specific precision/recall style metrics, latency/cost, fallback rate, user correction rate and audit of failed cases.

AI

When would you not use AI?

For deterministic, cheap, rule-based and audit-critical core logic. Example: the core pricing formula. AI is better for assistance, suggestions, document search or exception triage.

Good sentence: I would start read-only, prove value, add observability and evaluation, then introduce controlled write actions with human approval.

10. Mini system design: logistics pricing

Task

Design the architecture for a pricing tool where business users enter shipment parameters and receive auditable pricing results.

Clarifying questions

- Real-time quote or batch pricing?
- Which inputs: distance, carrier, capacity, SLA, fuel surcharge?
- Who can change rules, and is approval required?
- What latency is acceptable?
- Do we need audit logs and explainability?

Suggested architecture

- React admin UI with pricing forms and history.
- Node.js REST API for auth, validation and pricing orchestration.
- RDS for rules, configuration and audit data.
- SQS/EventBridge for batch recalculation and external carrier sync.
- CloudWatch metrics: quote latency, failure rate, stale data.
- AI only as assistant: rule explanation or anomaly suggestions, not the core formula.

Trade-off

If fast delivery is needed, start with a modular backend. If many teams and high scale justify it later, split service boundaries around pricing, shipment and carrier data.

11. Mini system design: shipment dashboard

Task

Build a dashboard showing current shipment status, delays and event history.

Data model

- Shipment: id, customer, route, status, ETA, carrier.
- ShipmentEvent: id, shipmentId, type, timestamp, source, payload.
- CurrentStatus can be denormalized for fast listing.
- Events can be immutable for audit.

API idea

- GET /shipments?status=&customer=&cursor=
- GET /shipments/{id}
- GET /shipments/{id}/events
- POST /shipments/{id}/notes or status override only with permission.
- Realtime: polling or SSE for server-to-client updates; WebSocket if two-way interaction is needed.

Pitfalls

- Virtualize large lists on the frontend.
- Use cursor pagination for large changing data.
- Idempotent event ingestion and duplicate handling.
- Show stale data clearly in the UX.

12. Mini system design: AI operations assistant

Task

An AI assistant answers business questions such as: Which shipments are at risk? What caused this pricing outlier?

Safe first version

- Read-only data access.
- Data filtered by user authorization.
- RAG over policies, shipment metadata and pricing rule descriptions.
- Tools: searchShipments, getPricingAudit, summarizeDelayReasons.
- Human approval before any write/export/email action.

Architecture sketch

- React chat/workflow UI.
- Node orchestrator: prompt templates, tool registry, auth context.
- MCP servers: logistics data resources and safe tools.
- Audit log: prompt metadata, tool calls, output, user approval.
- Evaluation set: common questions and expected factual answers.

Good trade-off sentence

For core operational decisions I would avoid autonomous write actions at first. I would start with explainable recommendations and keep a human in the loop.

13. Legacy PHP integration

Why it may come up

The JD mentions PHP as a plus, especially for maintaining or integrating legacy systems. The point is not to pretend to be a senior PHP specialist, but to show good migration and integration thinking.

Strangler fig approach

- Do not replace the whole legacy system at once.
- Start with an API facade or adapter layer.
- Build new features in Node/React while gradually replacing legacy data/processes.
- Use contract tests between old and new behaviour.
- Add monitoring, rollback and feature flags.

LEGACY

What would you do if you had to integrate with a legacy PHP service?

First understand the API/data contract, side effects and ownership. Then build an adapter layer with input/output validation, logging and contract tests.

LEGACY

When would you fully rewrite a legacy module?

Only if the business value, maintenance pain or security risk justifies it. Otherwise, gradual replacement is usually lower risk.

14. Coding task - what to expect

Likely format

For a full-stack technical interview, expect a smaller React component, API endpoint, data transformation, validation task, or pair-programming style discussion. The goal is clean thinking, not LeetCode tricks.

Coding checklist

- Clarify input and output.
- State your assumptions.
- Start with a simple working solution.
- Then handle edge cases: null, empty, duplicate, invalid input.
- Separate TypeScript types from runtime validation.
- Write testable functions; do not put all logic into the UI.
- Talk while coding: explain why you choose each step.

Mini practice tasks

- Filter and sort a shipment list in TypeScript.
- React form validation for pricing inputs.
- REST endpoint: POST /quotes with validation and error response.
- Async worker with retry and idempotency key.
- Normalize API response into UI models.

Important sentence: TypeScript helps at compile time, but external input still needs runtime validation.

15. Technical stories from your experience

Choose three stories

Do not use the same example for every question. Prepare one frontend story, one backend/realtime story, and one automation/AWS/AI-learning story.

Performance	React/API payload, bundle, rendering, monitoring.
Scalability	Node/WebSocket, Redis pub/sub, horizontal scaling.
Business value	Automation, less manual work, faster release or better visibility.
Failure	Detecting the issue, rollback, monitoring, postmortem learning.
Learning	AWS/AI training, demo projects, picking up a new stack quickly.

STAR template for technical answers

- Situation: what business/technical problem existed?
- Task: what was your responsibility?
- Action: which concrete technical decisions did you make?
- Result: what improved - latency, stability, developer speed, business visibility?

16. Quick question bank - Frontend

FE

Client state vs server state?

Client state is UI-specific: modal, selected row, local form input. Server state comes from outside: cache, stale data, refetching, errors and authorization.

FE

How would you handle permissions on the frontend?

Hide/disable UI for UX, but enforce security on the backend. A route guard is convenience, not a security boundary.

FE

What makes a component reusable?

Good abstraction level, explicit props, little or no business logic for UI primitives, or well-isolated domain components.

FE

How would you handle UI errors?

Inline validation, retry, error boundaries for render errors, user-friendly messages, request id for support. Use toasts only when appropriate.

FE

Accessibility basics?

Semantic HTML, keyboard navigation, focus management, label/input links, contrast. Use ARIA only when semantic HTML is not enough.

17. Quick question bank - Backend

BE

Authentication vs authorization?

Authentication answers: who are you? Authorization answers: what can you do? JWT/session only solves identity; permission checks are separate domain logic.

BE

How would you handle concurrency conflicts?

Optimistic locking with a version field, transactions, unique constraints, or idempotency keys depending on the domain problem.

BE

When do you need a transaction?

When multiple changes must succeed together. External API calls inside a DB transaction are usually risky; saga/outbox patterns may be better.

BE

What is the outbox pattern?

Write the domain change and the event into an outbox table in the same DB transaction. A separate publisher sends the event reliably.

BE

How would you handle external API failures?

Timeouts, retry policy, circuit breaker where needed, fallback, clear error mapping, alerting and manual replay when appropriate.

18. Quick question bank - Architecture

ARCH

Monolith or microservice?

A modular monolith is often faster and simpler at the beginning. Microservices are justified by independent deployment, ownership, scaling or clear domain boundaries.

ARCH

SQL or NoSQL?

Decide by query patterns, transactions and relationships. SQL for complex relational domains; NoSQL for high scale and known access patterns.

ARCH

Sync or async communication?

Sync when an immediate response is required. Async when the process is long, retryable, loosely coupled, or depends on an unreliable external system.

ARCH

How do you design scalable listing?

Indexes, filters, cursor pagination, limit, stable ordering, projections, caching, and possibly a read model.

ARCH

How do you handle audit?

Immutable event/audit log with actor, timestamp, old/new values, correlation id, retention and PII considerations.

19. Smart questions to ask them

Technical questions

- What is the biggest current technical pain in the React/Node stack?
- Is the current platform closer to a monolith or service-based architecture?
- Which AWS deployment model do you use: Lambda, ECS, EKS, or something else?
- What CI/CD and release process do you use for production?
- Do you have an observability stack: logs, metrics, traces, error tracking?
- For AI/MCP/agents, is there already a concrete use case or is it still exploration?
- What exact role does the PHP legacy system still play?
- How does a developer work with product and business stakeholders?

Good impression

Your questions should show that you care not only about technologies, but also about system ownership, release safety, business impact and maintainability.

20. Last-day review

90-minute plan

- 15 min: technical introduction out loud.
- 20 min: React/TypeScript questions.
- 20 min: Node/REST/error handling.
- 15 min: AWS/CI-CD/observability.
- 15 min: AI/MCP/agent security.
- 5 min: your questions for them.

Useful sentences

- I would start simple and add complexity only when the requirements justify it.
- I would validate input at the boundary and keep business logic separate from transport logic.
- For AI workflows, I would start read-only and add controlled actions later.
- I prefer measuring the problem first before optimizing blindly.
- I would design this around the business workflow, not around a fashionable architecture pattern.

Avoid

- Too many buzzwords without explanation.
- Solving everything with microservices.
- Using AI without control for core pricing/business rules.
- Listing AWS services without trade-offs.
- Saying that TypeScript removes the need for runtime validation.

Sources and notes

This material is based on the uploaded Redspher Full Stack Developer job description and its technical expectations: React, TypeScript, Node.js, AWS, REST APIs, Git/CI/CD, AI technologies, MCP, agents, automation, and optional PHP legacy integration.

MCP note

The MCP section is a practical interview-level summary of the concepts likely to matter: tools, resources, prompts, security, access control, audit and human approval.

AWS note

The AWS section is intentionally pragmatic: service choice, reliability, observability, safe deployment, IAM and automation. It is written for a full-stack technical interview, not for a cloud architect certification exam.

Important

This PDF is not an official Redspher interview question list. Its goal is to prepare you for the most likely technical themes based on the job description and common full-stack technical interviews.